CLOUD & DEVOPS

Service Discovery Visual Guide

Find services that never sit still — DNS, registries, and health checks.

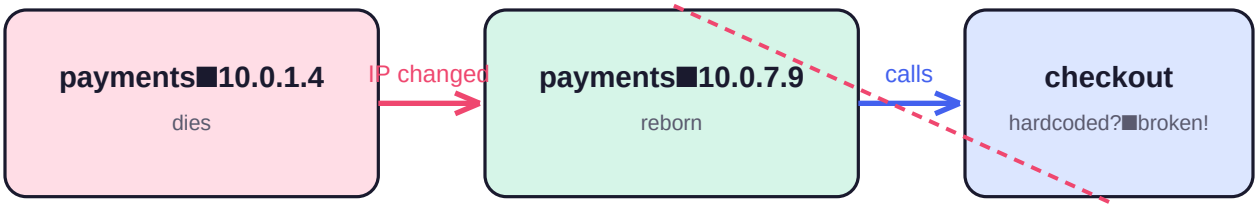
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Problem: Ephemeral IPs

Containers die and respawn with new addresses every minute.

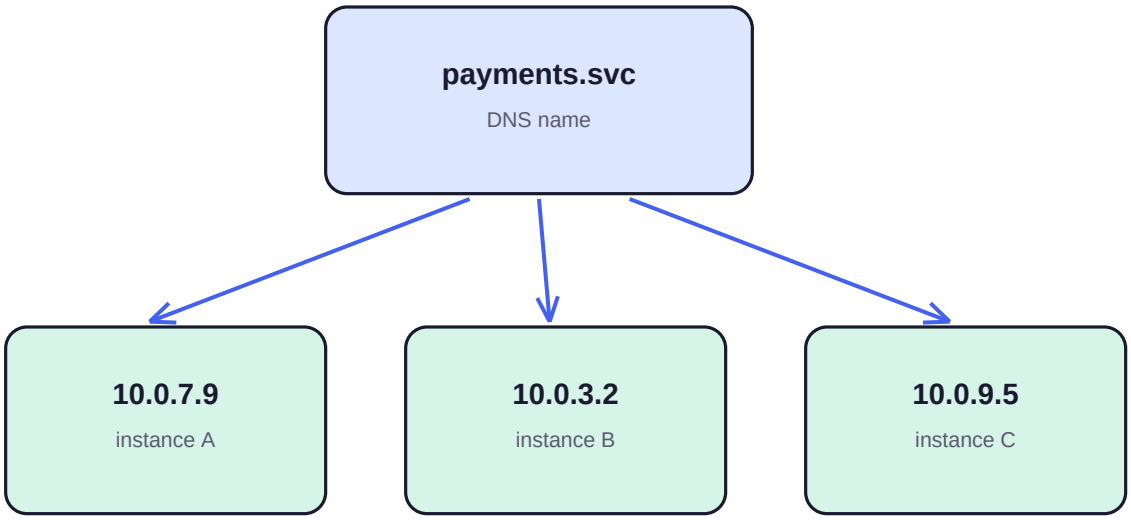


Hardcoded IPs and static config are the enemy — services must discover each other dynamically.

STEP 2/6

DNS-Based Discovery

The oldest trick: a name that resolves to healthy instances.



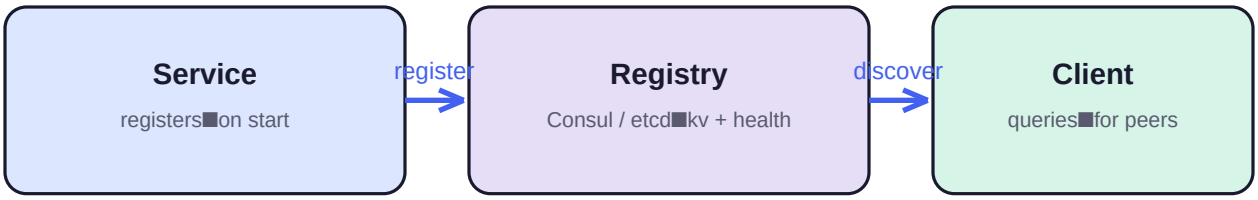
Kubernetes Services, AWS Cloud Map, Consul DNS: one stable name, many changing IPs.

DNS caching bites: low TTLs (seconds) or clients hold stale IPs. SRV records can also carry ports.

STEP 3/6

Registries: Consul & etcd

A strongly-consistent phone book for services.



- Registration: on start (or via agent); deregistration: on graceful shutdown + TTL expiry.
- etcd is the generic KV store (Kubernetes uses it); Consul adds DNS + health checking built in.
- Raft consensus: the registry itself is HA — 3–5 nodes, survives minority loss.

STEP 4/6

Client-Side vs Server-Side

Who picks the instance: the caller or a middleman?

CLIENT-SIDE

caller queries■registry, picks

- Smarter routing (latency-aware)
- Client needs discovery logic
- e.g. gRPC, Ribbon

SERVER-SIDE

caller hits LB,■LB picks

- Dumb clients, simple
- LB is a hop + failure point
- e.g. K8s Service, ALB

Service meshes blend both: sidecar proxies do client-side logic with server-side simplicity for the app.

STEP 5/6

Health Checking: Trust but Verify

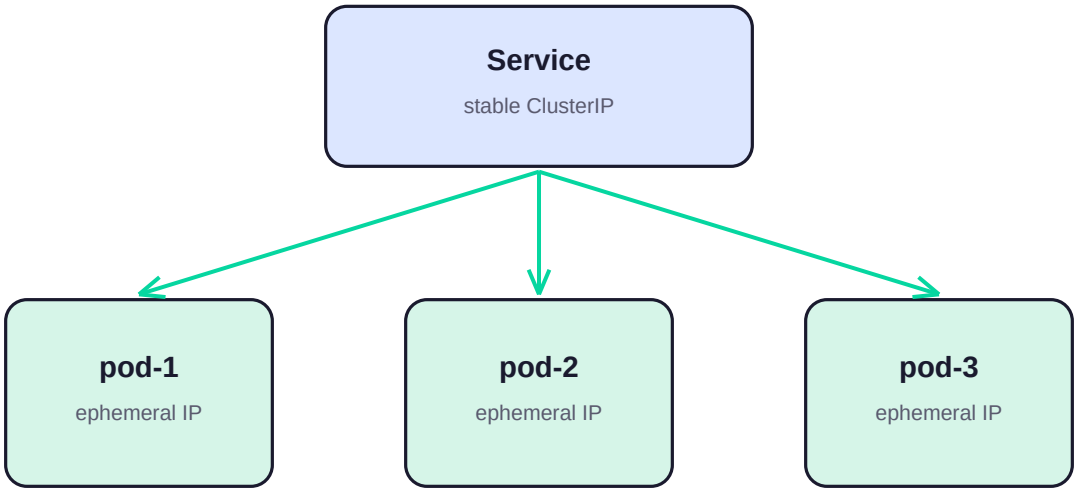
Discovery without health checks routes to corpses.

- Readiness gates traffic: failing readiness pulls the instance from discovery, no restart.
- Liveness restarts: only fail it when the process is truly wedged (deadlock), not on slow deps.
- Checks must be cheap and fast — a 5s health check is a 5s detection delay.

STEP 6/6

Discovery in Kubernetes

Services + endpoints: the built-in answer.



Endpoints controller tracks ready pods; kube-proxy / eBPF routes the Service IP to them.

Headless services (ClusterIP: None) return pod IPs directly — for stateful or client-side patterns.

STEP ✓I✓

Key Takeaways

Service Discovery — Remember

- Ephemeral IPs make static config fatal — services must discover each other dynamically.
- DNS gives stable names over changing IPs; watch TTL caching.
- Registries (Consul/etcd) are the HA phone book: register on start, health-check always.
- Client-side = smart callers; server-side = simple callers behind a LB; meshes blend both.
- Health checks (liveness/readiness/startup) keep discovery honest — cheap, fast, meaningful.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev